

uv: Cheat Sheet

uv is a fast Python package and project manager (a single tool that replaces pip, venv/virtualenv, pipx, and pyenv). Commands below are shell, run on macOS. Comments show real output.

Install (macOS)

```
# Official standalone installer (installs to ~/.local/bin)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Or via Homebrew
brew install uv

uv --version          # uv 0.11.17 (a33a629d6 2026-05-28 aarch64-apple-darwin)

# Keep uv up to date
uv self update        # standalone install only
brew upgrade uv       # if installed via Homebrew
```

Manage Python versions

uv downloads and manages standalone Python builds for you — no system Python needed.

```
# List installed + downloadable versions
uv python list
# cpython-3.14.5-macos-aarch64-none      <download available>
# cpython-3.12.13-...                   /Users/.../uv/python/.../python3.12
# cpython-3.9.6-macos-aarch64-none      /usr/bin/python3          (system)

# Install the latest stable CPython
uv python install          # latest stable
uv python install 3.14      # a specific minor -> newest patch
# Installed Python 3.14.5 in 1.33s
# + cpython-3.14.5-macos-aarch64-none (python3.14)

# uv-managed Pythons live under ~/.local/share/uv/python
# They do NOT shadow your system python3 unless you ask.

# Pin a version for the current directory (writes .python-version)
uv python pin 3.14

# Pin the global default uv uses where no local .python-version exists
uv python pin --global 3.14
# Pinned `~/.config/uv/.python-version` to `3.14`
uv python pin --global --rm # remove the global pin
```

The global pin sets which Python uv defaults to (for uv run, uv venv, new projects). It does not change your shell's python3 on PATH.

Make uv's Python your python / python3

macOS ships only python3 (the system build); there is no bare python, and a plain uv python install adds just a versioned python3.14 shim. To get python and python3 on PATH pointing at a uv build, install with --

default:

```
uv python install 3.14 --default
# warning: The `--default` option is experimental ...
# Installed Python 3.14.5
# + cpython-3.14.5-macos-aarch64-none (python, python3)
```

This writes `python`, `python3`, and `python3.14` into `~/.local/bin`. They win only if `~/.local/bin` precedes `/usr/bin` on your `PATH` (the `uv` installer puts it there). Open a new shell, then:

```
which python      # ~/.local/bin/python
python --version  # Python 3.14.5
```

Create a project

```
uv init uv-demo      # create a new project in a new directory
cd uv-demo
# Initialized project `uv-demo` at ../uv-demo

# Or initialise in the current (existing) directory
uv init
```

`uv init` scaffolds:

```
uv-demo/
├── .git/                # a git repo is initialised
├── .gitignore
├── .python-version      # 3.14
├── README.md
├── main.py
└── pyproject.toml
```

```
# pyproject.toml
[project]
name = "uv-demo"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.14"
dependencies = []
```

Run code

`uv run` executes inside the project's managed virtual environment, creating `.venv` and syncing dependencies on first use — no manual activation needed.

```
uv run main.py      # Hello from uv-demo!
uv run python --version  # Python 3.14.5 (the project interpreter)
uv run python -c "import sys; print(sys.executable)" # points at .venv
```

Add & remove dependencies

```
uv add requests          # runtime dependency
# Creating virtual environment at: .venv
# Resolved 6 packages ... Installed 5 packages
# + requests==2.34.2    (+ certifi, charset-normalizer, idna, urllib3)

uv add --dev pytest      # development dependency
# + pytest==9.1.1      (+ iniconfig, packaging, pluggy, pygments)

uv remove requests       # remove a dependency (and its now-unused deps)
# Uninstalled 5 packages

uv sync                  # install exactly what the lockfile says
                        # (e.g. right after cloning a project)

uv lock                  # re-resolve and refresh uv.lock
```

Pin a specific version

Pass a version specifier (PEP 508). Quote it so the shell doesn't interpret `<`, `>`, or `*`. Re-running `uv add` with a new spec just rewrites it.

```
uv add 'requests==2.31.0'    # exact pin
uv add 'requests>=2.30,<3'   # range
uv add 'requests~=2.31.0'    # compatible release (>=2.31.0, ==2.31.*)
uv add --dev 'pytest>=9,<10' # works for dev dependencies too

# Upgrade one package later without editing the spec
uv lock --upgrade-package requests
```

`uv add` updates three things together: `pyproject.toml`, the `uv.lock` lockfile, and `.venv`. Dev dependencies land under `[dependency-groups]`:

```
[project]
dependencies = [
    "requests>=2.34.2",
]

[dependency-groups]
dev = [
    "pytest>=9.1.1",
]
```

Coming from pip / venv

`uv` subsumes the classic `pip` + `venv` workflow. For project work prefer the high-level commands (`uv add/uv sync`); for a drop-in `pip` replacement, the `uv pip` subcommands mirror `pip`'s interface against a `uv`-managed environment.

Task	pip / venv	uv
Create a virtual env	<code>python -m venv .venv</code>	<code>uv venv</code>

Activate it	<code>source .venv/bin/activate</code>	not needed — <code>uv run</code> uses it
Add a package	<code>pip install requests</code>	<code>uv add requests</code>
Remove a package	<code>pip uninstall requests</code>	<code>uv remove requests</code>
Install from a file	<code>pip install -r requirements.txt</code>	<code>uv sync</code> (or <code>uv pip install -r ...</code>)
Pin/lock versions	<code>pip freeze > requirements.txt</code>	<code>uv.lock</code> (via <code>uv lock</code>)
Run in the env	<code>python script.py</code>	<code>uv run script.py</code>
Run a CLI tool once	<code>pipx run black</code>	<code>uvx black</code>

```
# Drop-in pip interface against the active/managed environment
uv pip install requests
uv pip freeze
uv pip install -r requirements.txt
```